MATLAB Phase Vocoder

Class: MUMT 307
Subject: Final Project Report
Name: Owen MacDonald
Date: December 12th, 2025

Introduction

As a student currently studying computer science, I immediately knew my final project would involve implementing an algorithm along with an interface. I chose to write this project in MATLAB because I was also taking COMP 350 (Numerical Computing) this semester, a course where learning MATLAB is essential. I was completely new to MATLAB, so I figured using it for this project would be a great way to learn it. As I was thinking about project ideas, I remembered learning about the vocoder in MUMT 250 last semester. I was going to propose this idea in class; however, I found out that a different group also had the same idea. That was when it was suggested that I implement a phase vocoder. I had no idea what a phase vocoder did and how it worked, but after a bit of research, it looked interesting so that is what I went with. This project consists of implementing a phase vocoder in MATLAB. The main objectives of this project are to learn what a phase vocoder does, how it works, implement it in MATLAB, and create a graphical user interface to make it easy and intuitive to use.

Methodology

Parameters

The first step of a phase vocoder is to initialize the parameters necessary for analysis and re-synthesis of the input sound. I decided to implement the algorithm as a function to make it easier to use it with different input parameters.

```
function y = phaseVocoder(x, tStretch, pitchShift, winLen, overlap)
```

Figure 1: Function header of the phase vocoder algorithm.

The first input parameter “x” is the given signal to analyze and re-synthesize. The second input parameter “tStretch” is the factor by which the duration of the sound sample “x” is extended or compressed. For example, if tStretch = 2, then the sound will be twice as long. If tStretch = 0.5, the sound will be half as long. The third input parameter “pitchShift” is the number of semitones that the pitch of the sample sound will shift. For example, if pitchShift = 5, the sound pitch will be shifted up by 5 semitones. If pitchShift = -2, the sound pitch will be shifted down by 2 semitones. The fourth input parameter “winLen” is the analysis window length, and the fifth and final input parameter “overlap” is the overlap percentage between windows during analysis.

```
N = length(x);

% Initialize parameters
pFactor = 2^(pitchShift/12); % Pitch factor
stretch = tStretch * pFactor; % Final stretch

H = round(winLen * (1-overlap)); % Sample hop length
strH = round(H * stretch); % Stretched hop length
win = hann(winLen, 'periodic'); % Periodic Hanning window for spectral analysis

numFrames = floor((N - winLen) / H) + 1; % Number of frames for STFT

y = zeros(numFrames * strH + winLen, 1); % Initialize output signal
```

Figure 2: Parameter initialization for STFT.

The variable "pFactor" is the factor by which the signal frequency changes when shifting pitchShift semitones. For example, shifting 12 semitones doubles the signal frequency and shifting –12 semitones halves it. One way to pitch shift a signal is to time stretch it by 1/pFactor, then resampling it at 1/pFactor times the original sample rate, which will be done after the time stretching. The variable "stretch" computes the final stretch factor that will be used in the algorithm. This gives us a way to independently change the signal pitch and duration. With these variables, we can compute the analysis hop length "H" which is used to compute the stretched hop length "strH" which we will use as the synthesis hop length. For the analysis window "win", a Hann window is used to reduce spectral leakage by eliminating the sharp discontinuities that cause energy to spread across adjacent frequency bins during the FFT. "numFrames" calculates the number of iterations for the STFT. Finally, "y" initializes the stretched output signal.

FFT

We are now entering the spectral analysis loop. The second step of a phase vocoder is to analyze the signal using the FFT.

```
% Spectral analysis loop
for i = 0:numFrames-1

    frame = x(i*H+1 : i*H+winLen) .* win; % Frame to analyze
    X = fft(frame);
    X = X(1:winLen/2+1); % Keep positive frequencies only
```

Figure 3: FFT of a frame of the input signal.

For every iteration of the loop, the FFT of a frame of size winLen is computed. We know that a size winLen FFT has winLen frequency bins with a spacing of fs/winLen Hz between them. A sinusoid will have the exact same frequency as a bin if an integer number of periods is completed within the FFT window.

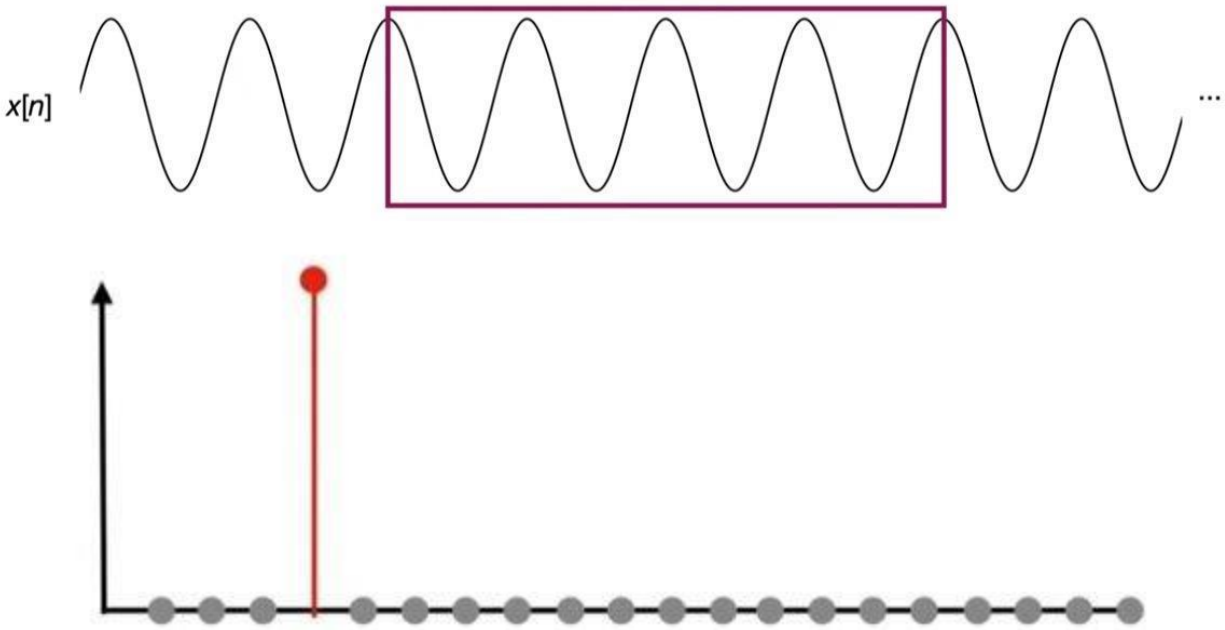


Figure 4: Window of a sinusoid with an integer number of periods (top) with its resulting FFT spectrum (bottom).

However, in most cases, the number of periods is not an integer, in which case the energy spreads to nearby frequency bins.

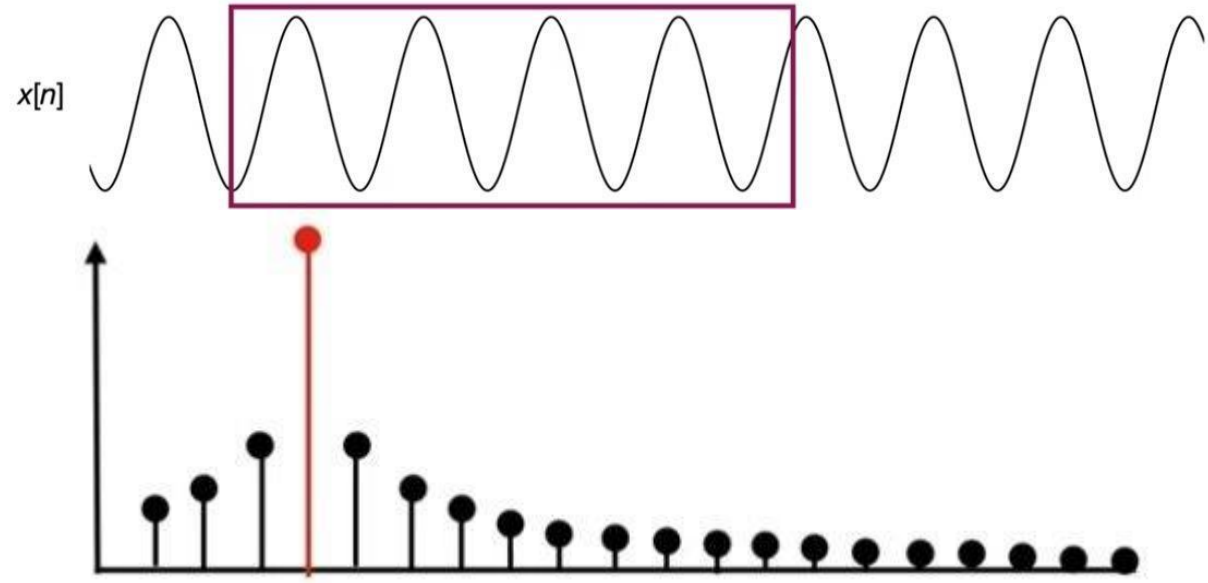


Figure 5: Window of a sinusoid with a non-integer number of periods (top) with its resulting FFT spectrum (bottom).

This is not what we want. Ideally, the FFT should find the exact frequency of this sinusoid component. However, there is a way to fix this. Notice how when the window has a non-integer number of periods, the phase is likely to change between frames. We can use this information to estimate the exact frequency of the sinusoid.

Adjusting Phase

The third step is to adjust the phase of each frequency component. First, we extract the magnitude and phase information from the FFT data.

```
% Convert complex FFT data to magnitude and phase
mag = abs(X);
phase = angle(X);
```

Figure 6: Magnitude and phase computed by FFT

We know the frequency in rad/s is the derivative of the phase. In discrete time, we can generalize the difference equation with H being the hop size and n being a sample point of the input signal "x".

$$\omega(t) = \frac{d\phi}{dt} \quad \omega[n] = \frac{\Delta\phi}{\Delta n}$$

$$\omega[n] = \frac{\phi[n] - \phi[n-H]}{(n) - (n-H)} = \frac{\phi[n] - \phi[n-H]}{H}$$

Figure 7: Derivative of the phase (top left), discretized derivative of the phase (top right), and discretized derivative of the phase with hop size H (bottom).

This gives us an equation to get the frequency of a sinusoid given its phase difference between hops of size H. From this, we can get an equation for the expected phase shift of a sinusoid over H samples given its bin number k.

$$\phi[n] - \phi[n-H] = H\omega[n]$$

$$\phi[n] - \phi[n-H] = \frac{2\pi kH}{N}$$

Figure 8: Phase shift between samples n-H and n given its frequency (top) and phase shift between samples n-H and n given its bin number (bottom).

Using these equations, we can subtract the expected phase shift from the actual phase shift of the sinusoid to get the phase remainder. The phase remainder can then be divided by H and added to the frequency at bin k to calculate the exact frequency of the sinusoid. For this equation to work, the phase remainder must be wrapped from the [0, 2pi] range to the [-pi, pi] range. We now have an equation to calculate the exact frequency of a sinusoid with the given FFT information.

$$\phi_r[n] = (\phi[n] - \phi[n-H]) - \frac{2\pi k H}{N}$$

$$\omega[n] - \omega_k = \frac{\text{wrap}(\phi_r[n])}{H}$$

$$\omega[n] = \frac{\text{wrap}(\phi_r[n])}{H} + \omega_k$$

Figure 9: Phase remainder (top), frequency deviation given phase remainder (middle), frequency given phase remainder and frequency at bin k (bottom)

However, this implementation of a phase vocoder uses a slightly different approach. Instead of calculating the exact frequency of a sinusoid, it keeps track of how the phase of the sinusoid changes with time using a phase accumulator defined before the STFT loop.

```
% Initialize phase accumulator to first frame
frame1 = x(1:winLen) .* win;
X1 = fft(frame1);
X1 = X1(1:winLen/2+1);
prevPhase = angle(X1);
phaseAcc = prevPhase;

expPhase = 2 * pi * (0:winLen/2)' * H / winLen; % Calculate expected phase advance
```

Figure 10: Phase accumulator initialization and expected phase advance for each frequency bin of the FFT.

This allows us to scale the phase advance by the stretch parameter defined in the first step. This way, for every frame of the STFT, a sinusoid phase is correctly updated from its previous phase given the FFT data of this frame. To time scale the signal, we simply scale the phase advance by the stretch factor so that the phase changes at a different rate.

```
% Calculate phase difference
phaseDiff = phase - prevPhase;
prevPhase = phase; % Update previous phase

% Remove expected phase advance
phaseDiff = phaseDiff - expPhase;

% Reduce to [-pi, pi] range
phaseDiff = phaseDiff - 2 * pi * round(phaseDiff / (2*pi));

% Update phase accumulator
phaseAcc = phaseAcc + (phaseDiff + expPhase) * (strH / H);
```

Figure 11: Adjusting phase of each sinusoid simultaneously.

This block of codes implements the equations mentioned earlier in this section. It first calculates the actual phase shift from the previous phase. Then it subtracts the expected phase shift from the actual phase shift. Then it wraps it to the [-pi, pi] range. Then the expected phase shift is added (instead of the expected bin frequency) and is scaled by the stretch factor. Then it updates the phase accumulator. Now the phase information is ready for the next step.

IFFT

The fourth step is to resynthesize the signal with the adjusted duration. To do so, the FFT must be reconstructed using the original magnitude along with the new phase.

```
    % Resynthesize time scaled output
    Y = mag .* exp(1i * phaseAcc); % Convert back to complex form for resynthesis
    Y = [Y; conj(Y(end-1:-1:2))]; % Mirror negative frequencies

    frameOutput = real(ifft(Y)) .* win;
    y(i*strH+1 : i*strH+winLen) = y(i*strH+1 : i*strH+winLen) + frameOutput;

end
```

Figure 12: IFFT of the adjusted FFT.

First, we convert the new data into its complex FFT form and store it in “Y”. Since the IFFT requires a full FFT for resynthesis, Y is mirrored as the second half was omitted at the start for analysis. The IFFT is then computed and overlap-added to the output signal variable “y”. Unfortunately, it is quite difficult to achieve perfect overlap-add when using different hop sizes for analysis and resynthesis. This means that there may be amplitude dips in the output signal if a small window overlap is used or if a very large time stretch is used relative to the overlap. More on this later in the “Challenges” section of the report.

Resampling

The fifth and final part of the phase vocoder algorithm consists of resampling the output signal at 1/pFactor times the original sample rate, that way the pitch shifting component of the phase vocoder is complete.

```
    % Resample at pFactor times the original sample rate
    [p, q] = rat(1/pFactor); % Returns two integers p and q to approximate 1/pFactor
    y = resample(y, p, q);

    % Pad or trim to exact desired length
    len = round(N * tStretch);

    if length(y) > len
        y = y(1:len);
    else
        y = [y; zeros(len - length(y), 1)];
    end

    % Scale to [-1, 1] range
    y = y / max(abs(y));

end
```

Figure 13: Resampling the output signal at 1/pFactor times the original sample rate.

To resample "y", the resample() function in MATLAB requires two positive integers "p" and "q". This function resamples its first input (in this case "y") at p/q times the original sample rate. This means we need to find two integers "p" and "q" to approximate 1/pFactor. This is taken care of by the MATLAB function rat(), which returns two integer arrays such that p./q approximates the input array. Finally, the output signal is resampled, trimmed or padded to the exact desired output signal length depending on the time stretch factor, and normalized to the [-1, 1] amplitude range. The variable "y" is the output of the phase vocoder function.

GUI

As mentioned in the introduction, part of this project was to create a graphical user interface to simplify the usage of the phase vocoder with different parameters. I decided to create an app using the MATLAB app designer because it's very simple to use and works great with MATLAB functions such as my phase vocoder function. The way the app designer works is that you have a design view where you can drag different objects from a library of objects and modify their properties, and then you have a code view where you can decide the function of each object. Here is the GUI I came up with for the program.

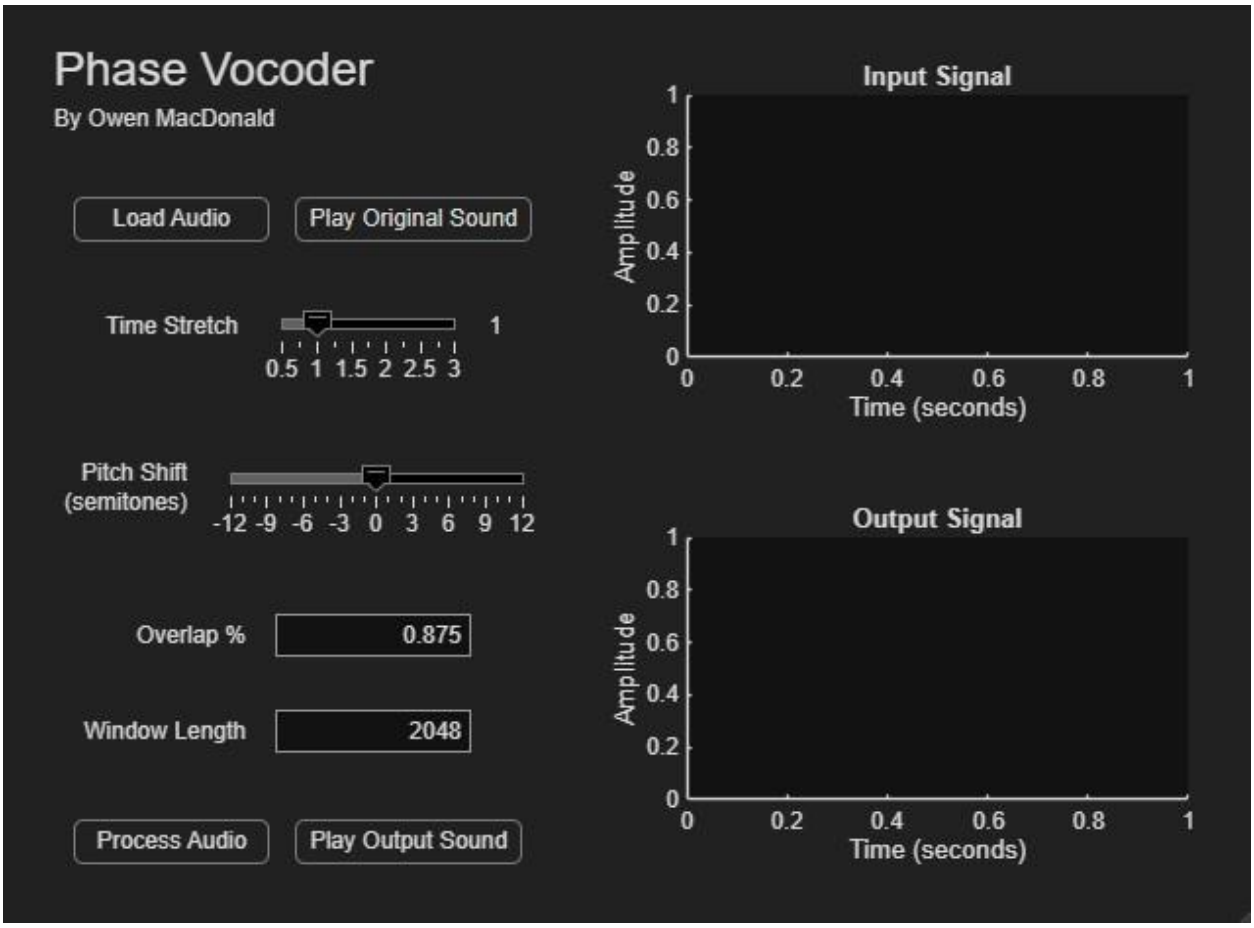


Figure 14: GUI for the MATLAB phase vocoder.

The first step is to load a .wav audio file. I could easily add more compatible file types, but only .wav files are compatible for now. Loading a .wav file will immediately display the input signal time-domain graph. You can then play the input signal if desired. The next step is to modify the input parameters of the phase vocoder function. The time stretch slider sets the time stretching factor to a value between 0.5 times the original length and 3 times the original length. I chose those bounds because stretching or compressing it any more than that either doesn't sound good or produces amplitude dips in the output signal. The pitch shift slider allows you to shift the pitch up or down an octave. I chose those bounds because it looks good, and I felt like it demonstrated the pitch shifting property of the program well enough. However, the bounds could have been way longer. The overlap % box has no limits, so the user can feel free to experiment with percentages higher than 100% and lower than 0% (I can't guarantee it will sound good though). The window length box also has no limits, so the user can also feel free to experiment with different window lengths (If the program does not like the suggested window length, it will not work). Lastly, the "Process Audio" button can be pressed to create an output signal with the givne parameters used as input in the phase vocoder function. This result is plotted in the output signal time-domain graph. Now, the "Play Output Sound" button may be pressed to play the resulting time stretched and pitch shifted sound. The user can play each sound as many times as they want in any order they want, that way they can compare both sounds if desired.

Demo

Here is a screenshot of the program with a loaded audio file and a processed output signal.

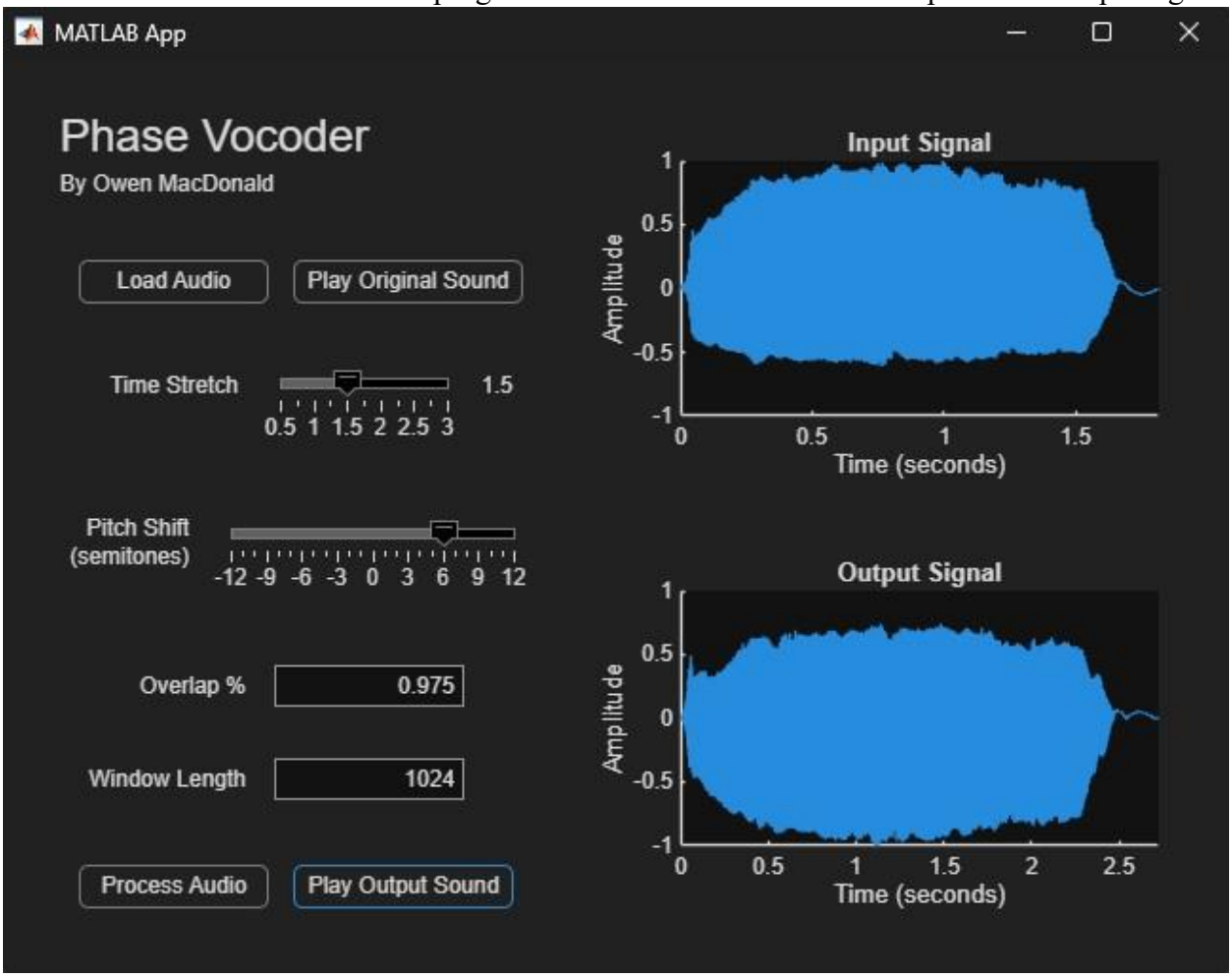


Figure 15: Final program demo with a loaded sound file (voice humming) time stretched to 1.5 times original length, pitch shifted up 6 semitones, an overlap % of 97.5% and a window length of 1024 samples.

It is a bit difficult to demonstrate this project with screenshots, so I recommend the user running the program for themself! If the user wants to exit this program, they may close the program by clicking the "X" at the top right corner of the GUI.

Challenges

During this project, I ran into several challenges. However, due to my extensive research on understanding the phase vocoder before starting to write the program, most issues I ran into weren't too much of a challenge to fix. The first issue I encountered was an amplitude modulation problem. After initially writing the algorithm, the output sound had a strange fluttering effect. It took me a while to realize that I wasn't accumulating the sinusoidal phases properly throughout the STFT iterations.

```
% FIRST CHALLENGE
% CHANGED TO: phaseAcc = phaseAcc + (phaseDiff + expPhase) * (strH / H);
% CHANGED FROM: phaseAcc = phaseAcc + expPhase * (strH / H) + phaseDiff;
```

Figure 16: First challenge current code compared to old code before modification.

Despite these modifications, the issue persisted. This made me review my code once more, which is where I realized that I was not properly initializing my phase accumulator. Instead of initializing each frequency component phase with the first FFT frame, I was initializing them at 0. This is wrong because it causes the phases to be out of sync with each other, resulting in a different sound.

```
% SECOND CHALLENGE
% CHANGED TO: frame1 = x(1:winLen) .* win;
%             X1 = fft(frame1);
%             X1 = X1(1:winLen/2+1);
%             prevPhase = angle(X1);
%             phaseAcc = prevPhase;
% CHANGED FROM: phaseAcc = zeros(winLen/2+1, 1); % Phase accumulator
%               prevPhase = zeros(winLen/2+1, 1); % Initial phase of 0
```

Figure 17: Second challenge current code compared to old code before modification.

After this modification, the issue still persisted. I spent a while going over the code, debugging, and experimenting, and nothing worked. I decided to do more research, which helped me understand that this issue had something to do with my window overlap. This fluttering effect occurs because of a low window overlap relative to the time stretch factor. This means that the more the duration of the input sound is extended, the higher the window overlap should be to reduce the rapid amplitude dips created by the windows during analysis and resynthesis.

Conclusion

In conclusion, this program implements a phase vocoder in MATLAB with time scaling and pitch shifting options, along with an intuitive GUI to allow easy parameter modification. Future improvements include implementing a real-time sound processing component where a sound is time scaled and pitch shifted in real time. However, this would require a completely new approach as the current phase vocoder function is not compatible with real-time audio. Another future implementation could be switching the time-domain graphs in the GUI to real-time spectrograms. Whenever a sound plays, its corresponding spectrogram would be displayed in real-time. This feature would be compatible with the current phase vocoder implementation, and I feel like it would be way more useful and relevant to this project than the time-domain graphs. Lastly, general improvements such as reducing the amplitude dips at high time stretches and low window overlap could be made to increase the range of each parameter, despite the amplitude dips being an effect of phase vocoders.

References

Understanding the phase vocoder

- Wikipedia contributors. (2025, June 30). Phase vocoder. Wikipedia. https://en.wikipedia.org/wiki/Phase_vocoder
- Phase Vocoder. (n.d.). https://ccrma.stanford.edu/~jos/sasp/Phase_Vocoder.html
- Bela Platform. (2020, November 17). 18: Phase Vocoder (part 1), C++ Real-Time Audio Programming with Bela [Video]. YouTube. https://www.youtube.com/watch?v=xGmRaTaBNZA
- Bela Platform. (2021, January 26). 19: Phase Vocoder (part 2), C++ Real-Time Audio Programming with Bela [Video]. YouTube. https://www.youtube.com/watch?v=2Esfl8uw-2U
- Bela Platform. (2021, February 17). 20: Phase Vocoder (part 3), C++ Real-Time Audio Programming with Bela [Video]. YouTube. https://www.youtube.com/watch?v=2p_jbl6Dyc
- Simcenter Testing. (2023, November 1). Leakage and window types (Hanning, flattop, uniform, exponential) [Video]. YouTube. https://www.youtube.com/watch?v=fC49T3LelE
- The phase Vocoder. (n.d.). https://caml.music.mcgill.ca/~gary/307/week11/node21.html

Implementing the phase vocoder in MATLAB

- Pitch Shifting and Time Dilation Using a Phase Vocoder in MATLAB. MATLAB Help Center. https://www.mathworks.com/help/audio/ug/pitch-shifting-and-time-dilationusing-a-phase-vocoder-in-matlab.html
- phase vocoder in Matlab. (n.d.). https://sethares.engr.wisc.edu/vocoders/phasevocoder.html
- resample. MATLAB Help Center. https://www.mathworks.com/help/signal/ref/resample.html
- rat. MATLAB Help Center. https://www.mathworks.com/help/matlab/ref/rat.html
- D. P. W. Ellis. (2002). A Phase Vocoder in Matlab. Web resource. https://www.ee.columbia.edu/~dpwe/resources/matlab/pvoc/

Implementing the GUI in MATLAB

- MATLAB App Designer. MathWorks. https://www.mathworks.com/products/matlab/app-designer.html